

QA Engineer Assignment

Objective

Build a maintainable QA automation solution demonstrating practical **UI, API, UI+API integration, and AI/RAG quality engineering**. The assessment evaluates framework design, business-risk coverage, backend validation, non-deterministic AI evaluation, retrieval/grounding, prompt-injection resistance and QA reasoning.

Applications Under Test

Parts 1-2: Practice Software Testing - Toolshop

Web: <https://practicesoftwaretesting.com>

API / Swagger: <https://api.practicesoftwaretesting.com/api/documentation>

Part 3: PDF Q&A; Chatbot (RAG) - Hugging Face

<https://huggingface.co/spaces/LovnishVerma/rag>

Candidates should explore the applications and select scenarios based on business value and risk.

PART 1 - UI AUTOMATION

Create a **Python + Playwright** UI automation framework and automate **5-8 meaningful business scenarios**. Coverage should demonstrate real workflows rather than isolated element checks.

Required Coverage

- Authentication/login, including valid and invalid behavior.
- Product search, filtering/sorting, product details and cart operations.
- Checkout or another meaningful multi-step business workflow.
- Positive and negative scenarios, user-facing error validation and business-rule validation.
- At least one end-to-end workflow spanning multiple pages/components.

Framework Expectations

- Reusable Page Object/component design with reliable locators and synchronization.
- Fixtures, setup/teardown, test-data handling and environment configuration.
- Meaningful assertions and failure evidence such as screenshots, traces or logs.
- Clean, maintainable project structure.

Part 1 Deliverables

- 5-8 automated UI scenarios with readable execution results.
- README explaining setup, execution, framework structure, assumptions and selected scenarios.

PART 2 - API TESTING + UI/API INTEGRATION

Using the Toolshop Swagger/API documentation, create a **Python + Pytest** API framework and automate **8-10 meaningful API scenarios**. Select endpoints based on business value and risk.

API Coverage

- Positive/negative scenarios; HTTP status, response-body and JSON/schema validation.
- Path, query and request-body parameter validation; authentication/token handling where applicable.
- Invalid, missing and boundary inputs; business-rule validation beyond status-code checks.
- Response-time checks and useful error reporting.

Framework Expectations

- Reusable API client/service layer, Pytest fixtures and parametrized/data-driven tests.
- Configuration separated from test logic; setup and cleanup where test data is created/modified.

Candidate may use products, authentication/users, cart, categories, favorites, invoices or other available resources.

UI + API Integration

Implement at least **one hybrid UI/API test**:

- **API -> UI**: prepare data/state through API, then execute or validate the business flow through UI.
- **UI -> API**: perform an action through UI and verify the resulting server-side state through API.

Explain why the selected approach improves speed, reliability, test-data control or backend validation.

Defect Identification

Document significant findings with title, reproduction steps, expected result, actual result and severity/priority recommendation.

Parts 1-2 Combined Deliverables

Deliverable	Expected Content
Automation project	Python project containing UI, API and hybrid automation.
Coverage	5-8 UI tests, 8-10 API tests and at least one UI+API integrated scenario.
Evidence	Readable report with passed/failed results and useful failure evidence.
README / findings	Setup, dependencies, configuration, execution, assumptions, framework explanation and defects.

PART 3 - RAG / LLM QUALITY TESTING

Primary application: <https://huggingface.co/spaces/LovnishVerma/rag>

Note: If the public application is unavailable, unstable, rate-limited or lacks a needed capability, use another public AI/RAG chatbot or a local/open-source RAG application and document the alternative and assumptions.

Candidate Task & Ground Truth

Create a reusable **Python + Pytest AI evaluation framework**. Use small documents with known facts and build **10-12 meaningful scenarios** containing the question, expected facts/behavior, expected source(s), risk level and PASS/FAIL conditions.

- Include current vs archived/conflicting information, multi-document/section questions and numerical/rulereasoning.
- Include absent-information, ambiguous/insufficient-evidence and source-grounding cases.
- Include a document containing malicious/misleading instructions for indirect prompt-injection testing.

Key AI Quality Metrics

Metric	Evaluation Focus
Correctness	Factual/conclusion accuracy against defined ground truth.
Faithfulness	Whether claims are supported by retrieved/uploaded evidence.
Relevance	Whether the answer directly addresses the question.
Hallucination	Unsupported facts, values, rules or conclusions.
Consistency	Semantic stability across repeated executions.
Grounding	Traceability of important claims to reliable source context.
Safety	Resistance to malicious instructions and unauthorized behavior.

Use measurable evaluation rather than exact-string matching alone. Define scoring/PASS-FAIL thresholds and explain the evaluation logic.

RAG Retrieval & Grounding

- **Retrieval relevance/completeness:** relevant passages are retrieved and required evidence is present.
- **Source/version selection:** current/authoritative evidence is preferred over outdated or conflicting content.
- **Grounding/citation:** generated claims and exposed references genuinely match retrieved evidence.

Include at least one case showing that **faithfulness is not the same as correctness**; an answer may faithfully use outdated evidence yet reach an incorrect conclusion.

Document Processing Testing

Validate or describe checks across **Upload -> Extraction -> Chunking/Indexing -> Retrieval -> Answer**. Cover processing failures, partial extraction, tables/structured data, duplicates/overlap, large or multi-page documents, conflicting versions and chunk-boundary content. Show how you would isolate whether a defect originated in processing, retrieval or generation.

PART 4 - PROMPT INJECTION, AGENTIC & RELEASE QUALITY

Prompt Injection & AI Safety

- **Direct injection:** attempts to override intended behavior through the user prompt.
- **Indirect injection:** malicious instructions embedded inside an uploaded document and later retrieved as context.

- Attempts to ignore trusted sources, fabricate a preferred answer, expose hidden/system information or answer outside authorized document context.
- Verify malicious retrieved text is treated as untrusted content rather than an instruction.

Agentic / Tool-Based Testing

If the application exposes tools, retrieval actions or intermediate steps, automate them. Otherwise create a small mocked agent/tool layer or equivalent automated tests.

- Correct retrieval/tool selection; unnecessary/repeated calls; timeout/service failure; missing/invalid tool response.
- When sufficient evidence cannot be obtained, the AI should not fabricate a confident answer.

Regression & AI Quality Gate

Design the framework so the same dataset can compare **Prompt V1 vs Prompt V2, Model A vs Model B, or retrieval/configuration changes**. Report metric-level improvements and regressions.

Define measurable release criteria: minimum correctness/faithfulness, maximum hallucination rate, minimum retrieval/grounding quality and **zero critical prompt-injection or safety failures**. A better average score must not hide a critical regression.

Part 3 Framework & Deliverables

- Reusable Python/Pytest framework with fixtures, parametrization and clean configuration.
- External ground-truth dataset containing 10-12 scenarios with expected evidence and pass/fail expectations.
- Automated evaluation of Correctness, Faithfulness, Relevance, Hallucination, Consistency, Grounding and Safety.
- RAG/retrieval, document-processing, direct/indirect prompt-injection and agent/tool failure coverage.
- Readable metric-level PASS/FAIL report and README covering setup, execution, assumptions, thresholds, limitations and alternatives.

Overall Evaluation Criteria

Area	Weight
Python & framework architecture	20%
UI automation quality	15%
API + UI/API integration quality	20%
LLM quality evaluation	20%
RAG retrieval, grounding & document processing	15%
Prompt injection, safety, agentic testing & reporting	10%

Submission Guidelines

- Submit as a Git repository or ZIP with clear instructions so another engineer can execute the solution.
- Do not include credentials, secrets, tokens or private information. State assumptions and limitations.
- Be prepared to explain framework architecture, scenario selection, test strategy, thresholds/evaluation logic and code.

Evaluation emphasis: quality and reasoning behind the automation, not simply the number of test cases implemented.